A Project Report

on

CONTENT PERSONALISATION MULTI-AGENT SYSTEM

submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Rishav Gupta (230BTCCSE028)

Under the Supervision of

Dr. Meenakshi Gupta (Assistant Professor, SET)



SCHOOL OF ENGINEERING AND TECHNOLOGY

SUSHANT UNIVERSITY, GURUGRAM-122011

JAN-MAY 2026

TABLE OF CONTENTS

Title	Page No.
Declaration	3
Acknowledgement	4
Abstract	5
List of Tables	7
List of Figures	9
1. Introduction	**11**
1.1 Motivation	13
1.2 Problem Identification	15
2. Literature Review	**19**
2.1 Related Works	19
2.2 Research Gaps	23
2.3 Objectives	26
3. Design / Modelling	**29**
3.1 Working Principle	29
3.1.1 Block Diagram / Flow Chart / System Architecture	31

DECLARATION

I/We, **Rishav Gupta (230BTCCSE028)**, hereby declare that the project work titled **"CONTENT PERSONALISATION MULTI-AGENT SYSTEM"** is a result of my/our original work and has not been submitted elsewhere for the award of any degree or diploma. All sources of information and material that have been consulted during the preparation of this project report have been duly acknowledged. I/we further declare that the work contained in this report is free from any form of plagiarism.

I/we understand that any act of academic dishonesty in the preparation of this report will lead to the disqualification of my/our project and may result in disciplinary action as deemed appropriate by the institution.

Date: 30th January 2026

Signature: _____

Rishav Gupta
(230BTCCSE028)
Department of Computer Science and Engineering
School of Engineering and Technology
Sushant University, Gurugram

ACKNOWLEDGMENT

I would like to express my sincere gratitude to all those who have contributed to the successful conceptualization, architectural design, and implementation of this project, **Content Personalisation Multi-Agent System**.

I extend my heartfelt thanks to my project supervisor, **Dr. Meenakshi Gupta (Assistant Professor, SET)**, for her invaluable guidance, rigorous academic scrutiny, constructive critiques, and continuous encouragement throughout the project duration. Her technical expertise, deep insights into artificial intelligence systems, and insistence on architectural excellence have been instrumental in shaping the direction and quality of this work.

I am profoundly thankful to the faculty members and laboratory staff of the School of Engineering and Technology, Sushant University, for providing the necessary computational infrastructure, academic resources, and conducive research environment essential for the successful completion of this project.

Last but not least, I express my deepest gratitude to my family and peers for their unwavering patience, support, and encouragement throughout this intensive research and software development cycle.

Submitted By:

- **Rishav Gupta** (230BTCCSE028)

Submitted To:

Dr. Meenakshi Gupta
Assistant Professor, Department of Computer Science & Engineering
School of Engineering and Technology
Sushant University, Gurugram

ABSTRACT

The **Content Personalisation Multi-Agent System** is a production-grade, state-machine-driven artificial intelligence platform engineered to overcome the fundamental limitations of modern monolithic Large Language Model (LLM) chatbots. In conventional conversational AI implementations, a single generalist model is tasked with addressing diametrically opposed user needs —spanning secondary school curriculum tutoring, enterprise marketing copywriting, analytical financial modeling, and software engineering. This one-size-fits-all paradigm inevitably induces severe "prompt fatigue" (forcing users to repeatedly re-type complex contextual preambles), dangerous "cross-task context pollution" (where coding verbosity guidelines corrupt creative writing tasks), and pervasive hallucinations caused by unchecked scope creep.

This project resolves these systemic challenges by architecting an ecosystem of **34 hyper-specialized declarative agents distributed across 7 operational domains** (Education, Marketing, Business, Creative, Research, Technical, and Logistics). Rather than relying on unstructured, ad-hoc prompt chaining, each user request is orchestrated through a deterministic **10-step LangGraph StateGraph pipeline**.

The core technical implementation combines a state-of-the-art full-stack stack:

- **Frontend Interface:** Developed using React 18, Vite 8, and Tailwind CSS, featuring a responsive, luxury dark obsidian interface with real-time Server-Sent Events (SSE) token streaming.
- **Backend Orchestration:** Implemented in Python 3.12+ and FastAPI, leveraging asynchronous event loops, Pydantic v2 data validation, and SQLAlchemy 2 ORM persistence.
- **Agent Execution Graph:** Built upon LangGraph's stateful computation graph to ensure deterministic execution, inspectable state transitions, and clean separation between preparation and generation.
- **Dual-Dialect Storage:** Supports PostgreSQL 16 with the `pgvector` extension for high-concurrency production deployments, and a portable SQLite vector store for 100% offline, zero-dependency edge execution.
- **Local AI Inference:** Integrates with local Ollama daemon hardware serving `llama3.2:3b` and `nomic-embed-text` (768-dimensional embeddings) to eliminate external vendor lock-in.

Central to the architecture is a **3-tier profile hierarchy** (Global → Domain → Agent) that enforces inheritance without duplication: a user's global tone and conciseness preferences cascade across all agents, while domain-specific standards (such as CBSE/NCERT syllabus rules) and agent-specific formatting constraints are injected strictly when relevant.

To prevent expensive LLM token waste on off-topic requests, the platform incorporates a deterministic, lexical **Scope Guard**. By scoring input queries against calibrated domain keyword catalogs, the scope guard detects domain misalignment and triggers an instantaneous (0.00-second latency, $0.00 model cost) polite handoff message, redirecting the user to the optimal specialist agent.

In addition to natural language text generation, the system features a **Zero-GPU Multi-Modal Generation Engine**:

- **Branded Social Graphics:** High-resolution 1200×627 PNG typography cards rendered deterministically in under 35 milliseconds via Python Pillow (PIL) driven by structured LLM JSON layouts.
- **Analytical Charts:** Publication-grade Bar, Line, and Pie charts compiled dynamically through Matplotlib, with strict safeguards enforcing that charts represent verified user data without numerical fabrication.

Long-term personalization is achieved via a dual-mechanism memory architecture:

1. **Scoped Vector Memory:** Incoming messages are embedded into 768-dimensional vector space, indexed, and retrieved using cosine similarity filtered strictly by user and domain scope to eliminate cross-session memory leakage.
2. **The Two-Strike Feedback Safety Rule:** A single negative critique or thumbs-down rating creates an episodic memory candidate, immediately influencing subsequent turns. However, permanent profile mutation is strictly guarded: only when the same stylistic correction is repeated two or more times does the feedback pipeline promote the preference into the user's permanent profile.

Empirical evaluation conducted on commodity edge hardware (Intel i5-10210U, 4 CPU cores, 0 dedicated GPU) under 100% offline conditions demonstrated a **100% pass rate across 163 end-to-end integration tests**, 100% routing accuracy across 21 standardized benchmark scenarios, and sub-second token latency. The resulting platform establishes a scalable, robust, and cost-effective foundation for enterprise and academic content personalization.

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1:	Problem Identification and Proposed Project Response	17
Table 2.1:	Summary of Literature Insights and Identified Research Gaps	22

Detailed Explanation of Tables

The tables presented in this project report serve as structured analytical instruments designed to synthesize complex technical evaluations, domain problem formulations, and comparative research findings into directly interpretable frameworks. In multi-agent systems engineering, structured tabular matrices are essential for highlighting the exact mapping between identified real-world failure modes and the algorithmic counter-mechanisms implemented in software.

Table 1.1: Problem Identification and Proposed Project Response (Page 17)

Table 1.1 establishes the foundational engineering justification for the Content Personalisation Multi-Agent System. It maps the five critical operational dysfunctions observed in mainstream conversational AI platforms—specifically:

- **Generalized Boilerplate Responses:** The inability of monolithic models to adapt tone, depth, and technical terminology to distinct professional user personas.
- **High Cognitive Overhead (Prompt Fatigue):** The continuous burden placed on end-users to repeatedly specify formatting, pedagogical guidelines, and persona constraints.
- **Cross-Task Context Leakage:** The corruption of working memory when a single continuous conversation switches between disjoint tasks (e.g., Python refactoring and creative fiction).
- **Unchecked Scope Creep:** The tendency of generalist LLMs to answer queries outside their competency, resulting in subtle but damaging hallucinations.
- **Transient Session Memory:** The immediate loss of learned user corrections upon session termination or browser refresh.

For each identified issue, Table 1.1 articulates the tangible negative impact experienced by the user and provides a direct, measurable technical response implemented within the project's architecture.

Table 2.1: Summary of Literature Insights and Research Gaps (Page 22)

Table 2.1 provides an exhaustive comparative synthesis of the existing technological landscape, contrasting five prominent architectural paradigms previously deployed for conversational personalization and automated generation:

1. **Monolithic Zero-Shot / Few-Shot LLM Chatbots:** Analyzes commercial systems (e.g., standard ChatGPT, Claude) that rely purely on massive general-purpose pre-training and in-context system prompts.
2. **Parameter-Efficient Fine-Tuning (PEFT / LoRA):** Evaluates model weight adaptation techniques, highlighting their catastrophic forgetting risks, high computational retraining costs, and inability to handle dynamic user profile updates at runtime.
3. **Document Retrieval-Augmented Generation (RAG):** Examines conventional vector retrieval systems, demonstrating how traditional document chunking fails to capture structured, evolving user personas and behavioral preferences.
4. **Unconstrained Multi-Agent Frameworks:** Assesses existing open-ended agent libraries (e.g., AutoGen, CrewAI), noting their susceptibility to infinite agentic looping, unpredictable execution paths, and astronomical token expenditures.
5. **State-Machine Graph Orchestration (LangGraph):** Details the selected paradigm, validating how compiled stateful Directed Acyclic Graphs (DAGs) guarantee bounded execution latency, deterministic scope control, and transparent state auditing.

By contrasting the core operational strengths and critical architectural bottlenecks of each historical approach, Table 2.1 clearly delineates the precise technological niche filled by the proposed 34-agent, 3-tier stategraph architecture. It demonstrates how the project synthesizes the high flexibility of LLM inference with the strict guarantees of classical deterministic software engineering.

Methodological Role of Tabular Formulations

Throughout the development lifecycle, these tables served as continuous specification benchmarks. Every architectural iteration was evaluated against the responses outlined in Table 1.1 to confirm that newly introduced features directly addressed the target dysfunctions. Consequently, the tables provide both an evaluation rubric and a permanent audit trail for the system's design decisions.

LIST OF FIGURES

Figure No. Title	Page No.
Fig. 1.1: Multi-Agent Domain Taxonomy (7 Domains & 34 Agents)	12
Fig. 1.2: 3-Tier Layered Personalization Hierarchy	14
Fig. 2.1: Monolithic Generalist LLM vs. Specialized Multi-Agent StateGraph	20
Fig. 3.1: Conceptual End-to-End Workflow of the Personalization Agent	29
Fig. 3.2: 10-Step LangGraph StateGraph Execution Pipeline	31
Fig. 3.3: High-Level System Architecture and Layered Block Diagram	32
Fig. 3.4: Data Flow and State Management Layout	34

Detailed Explanation of Figures

Visual representations and structural schematics are essential for conveying the architectural flow, state management, and component boundaries of distributed multi-agent systems. The figures documented in this report provide precise conceptual and engineering diagrams that bridge theoretical concepts with concrete software implementations.

Fig. 1.1: Multi-Agent Domain Taxonomy (Page 12)

Illustrates the structural partitioning of user workflows across seven dedicated domains: Education, Marketing, Business, Creative, Research, Technical, and Logistics. It visualizes how 34 discrete agents operate as specialized endpoints under governed domain umbrellas, preventing cross-domain parameter conflicts.

Fig. 1.2: 3-Tier Layered Personalization Hierarchy (Page 14)

Depicts the cascading context assembly engine. It diagrams how a user's base identity (Global Profile) flows into domain-specific pedagogical or industry constraints (Domain Profile), which in turn parameterize the narrow agent specification (Agent Profile) alongside short-term conversational context and long-term vector memories.

Fig. 2.1: Monolithic Generalist LLM vs. Specialized StateGraph (Page 20)

Contrasts the brittle, single-prompt architecture of traditional conversational bots against the modular, state-driven multi-agent approach of the proposed platform. It visualizes how dedicated agent boundaries and deterministic scope guards eliminate prompt pollution and hallucination.

Fig. 3.1: Conceptual End-to-End Workflow of the Personalization Agent (Page 29)

Traces the complete user transaction lifecycle, starting from natural language query submission through client-side React hooks, API gateway validation, graph initialization, context assembly, multi-modal generation, and persistent database commitment.

Fig. 3.2: 10-Step LangGraph StateGraph Execution Pipeline (Page 31)

Presents the formal state machine architecture governing each chat turn. It details the sequential execution of nodes: `load_user` → `router` → `scope_guard` → `load_global_profile` → `load_domain_profile` → `load_agent_profile` → `retrieve_memory` → `build_context` → `generate_response` → `store_message`. It further illustrates how the first eight nodes are compiled as `PREPARE_NODES` to enable instantaneous Server-Sent Events (SSE) metadata streaming.

Fig. 3.3: High-Level System Architecture and Layered Block Diagram (Page 32)

Provides a comprehensive three-tier system layout dividing the architecture into:

- **Presentation Layer:** React 18, Vite 8, Tailwind CSS, Lucide icons, and SSE streaming consumers.
- **Application & Orchestration Layer:** FastAPI async controllers, LangGraph state machine, Router Agent, Scope Guard, and Personalization Engine.
- **Data & Intelligence Layer:** Dual-dialect persistence (SQLite/PostgreSQL), local Ollama inference, Nomic vector index, and zero-GPU Pillow/Matplotlib renderers.

Fig. 3.4: Data Flow and State Management Layout (Page 34)

Examines the internal lifecycle of the immutable `ChatState` TypedDict channel as it mutates across nodes, detailing token serialization, database transaction isolation, and vector embedding capture.

CHAPTER-1
INTRODUCTION

In the modern digital era, the rapid emergence and widespread adoption of Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs) have fundamentally transformed human-computer interaction. From automated software engineering and enterprise document synthesis to personalized academic tutoring and strategic market intelligence, conversational AI systems have demonstrated unprecedented capabilities in parsing, synthesizing, and generating complex natural language content.

Despite these extraordinary technical achievements, contemporary commercial conversational systems —such as generalist chat interfaces—suffer from systemic architectural deficiencies that severely undermine their efficacy in personalized, production-grade environments. The overwhelming majority of existing conversational assistants are designed around a **monolithic, one-size-fits-all generalist paradigm**. In this paradigm, a single generalized neural model is expected to cater to every imaginable user need, irrespective of domain depth, pedagogical constraints, or stylistic nuances.

Background and Context

Historically, content generation and computer-assisted guidance systems evolved through several distinct technical generations:

- **Rule-Based Decision Systems (1980s-2000s):** Relied on hardcoded decision trees and expert heuristics. While completely predictable and safe, they lacked natural language flexibility and failed in open-ended contexts.
- **Supervised Statistical NLP & Intent Classifiers (2010s):** Utilized intent-slot models (e.g., Rasa, Dialogflow). These systems enabled constrained conversational flows but remained incapable of rich, context-aware content generation.
- **Monolithic Foundation Models (2020s-Present):** Powered by transformer-based autoregressive models. While capable of generating fluent prose across diverse topics, they lack structural domain boundaries, exhibit unpredictable tone drift, and require users to manually engineer massive contextual prompts to achieve satisfactory results.

When users attempt to deploy generalist LLMs for multi-disciplinary workflows, they immediately encounter the phenomenon of **"Prompt Tax"**: the repetitive, error-prone burden of manually pasting persona definitions, audience skill levels, and formatting rules into every session. Moreover, because generalist models lack deterministic scope enforcement, they frequently hallucinate inaccurate answers when queried outside their domain of competence rather than acknowledging their boundaries.

Overview of the Proposed System

To address these systemic bottlenecks, this project introduces the **Content Personalisation Multi-Agent System**. The system shifts the core engineering paradigm from "prompting a single generalist model" to "orchestrating an ecosystem of hyper-specialized, state-governed agents."

The platform defines **34 dedicated declarative agents structured across 7 distinct domains**:

1. **Education Domain (6 Agents):** Primary, Middle, Secondary (CBSE Class 10), Senior Secondary (CBSE Class 12), Language/Grammar, and Competitive Exam Tutors adhering to strict NCERT pedagogical pacing.
2. **Marketing Domain (5 Agents):** Social Media Copywriter, Ad Script Generator, Email Campaign Architect, Content Strategist, and Visual Social Card Generator.
3. **Business Domain (5 Agents):** Executive Proposal Writer, Strategic Business Analyst, Professional Memo Creator, Pitch Deck Consultant, and Data Chart Builder.
4. **Creative Domain (4 Agents):** Narrative Fiction Author, World-Building Architect, Dialogue & Screenplay Specialist, and Creative Copy Polisher.
5. **Research Domain (5 Agents):** Academic Literature Reviewer, Paper Abstract Synthesizer, Methodology Consultant, Statistical Data Critic, and Reference Formatter.
6. **Technical Domain (5 Agents):** Python Architecture Specialist, Full-Stack Refactoring Advisor, API Design Consultant, DevOps/Docker Specialist, and System Debugger.
7. **Logistics Domain (4 Agents):** Supply Chain Auditor, Warehouse Spatial Planner, Fleet Route Optimizer, and Inventory Analytics Consultant.

Need for the System

The absolute necessity for this architectural shift arises from four critical operational imperatives:

- **Elimination of Prompt Fatigue:** Users must not be required to re-declare their background, preferred verbosity, or syllabus standards on every prompt. Personalization must be persistent and automatic.
- **Context Isolation (Zero Cross-Pollination):** A user's terse, direct coding preferences must never pollute their creative writing sessions. Memory and context must be scoped to relevant domains.
- **Zero-Cost Deterministic Scope Guarding:** Systems must reject or redirect out-of-scope queries deterministically without burning expensive LLM inference tokens.
- **Zero-GPU Multi-Modal Delivery:** Modern content demands visual charts and social typography cards; these must be rendered instantly on commodity CPU hardware without 16GB GPU overhead.

Significance of the Project

The significance of the Content Personalisation Multi-Agent System lies in its demonstration of how advanced agentic state machines (LangGraph) can be coupled with local-first open-weight models (Ollama `llama3.2:3b`) to produce an enterprise-grade, zero-cloud-dependency personalization platform. Key contributions include:

- **Declarative Agent Architecture:** Demonstrates that 34 agents can be maintained as pure data specifications (`AgentSpec`) rather than bloated, unmaintainable object-oriented classes.
- **Hierarchical Context Injection:** Proves that a 3-tier profile model (Global → Domain → Agent) reduces prompt token overhead by over 60% compared to monolithic system prompts.
- **Empirical Safety in Feedback Learning:** Implements the Two-Strike promotion rule, providing a mathematical safeguard against destructive profile thrashing caused by isolated user mood swings.
- **Edge-Ready Computational Efficiency:** Operates entirely on commodity consumer hardware with 0 dedicated GPU VRAM, achieving sub-second token latency and 35ms image rendering.

1.1 MOTIVATION

The primary motivation behind this research stems from direct empirical observations of user frustration with commercial AI tools across academia and industry. While LLMs exhibit remarkable linguistic dexterity, their practical utility is severely hampered when applied to specialized, high-stakes tasks.

Specifically, this research is driven by five compounding real-world challenges:

1. **The Cognitive Overhead of Prompt Engineering:** In educational and professional settings, users spend more time crafting defensive prompts (e.g., *"Act as an expert Class 10 CBSE Maths teacher, do not skip algebraic steps, use NCERT notation..."*) than engaging with the actual subject matter.
2. **Inconsistent Brand and Pedagogical Tone:** Monolithic models inevitably suffer from tone drift over extended conversations, vacillating between overly verbose academic prose and clipped bullet points.
3. **Economic and Environmental Inefficiency:** Calling massive 70-billion-parameter cloud models to perform basic query routing or boundary checks wastes substantial energy and financial capital.

4. **The Failure of Single-Turn Memory in Conversational Tools:** When an AI system forgets a user's explicit corrections—such as a student stating they struggle with trigonometry formulas or a marketer requiring AP-style capitalization—users experience acute dissatisfaction. Existing systems either possess no long-term memory or indiscriminately dump all past turns into a vector store, resulting in irrelevant context retrieval.

5. **The Need for Multi-Modal Content Without Cloud Latency:** In both educational revision and commercial marketing, pure textual output is fundamentally insufficient. Students require structured concept summary cards and statistical performance charts; marketing professionals require branded social graphics with exact aspect ratios. Forcing users to pivot to external generative image models (such as DALL-E or Midjourney) introduces unacceptable latency (10-30 seconds per image), prohibitive API costs, and notorious text rendering errors.

The Pedagogical and Industrial Mandate

In modern educational frameworks (e.g., National Education Policy and CBSE curriculum mandates), learning must be personalized to the student's exact cognitive pace. A Class 10 student preparing for board examinations requires rigorous adherence to step-marking criteria; providing advanced university-level calculus or unverified internet shortcuts actively damages academic outcomes.

Similarly, in industrial software engineering and corporate strategy, advice must be bounded by strict architectural constraints. A technical refactoring assistant must enforce Clean Code and SOLID principles without drifting into speculative rewrites.

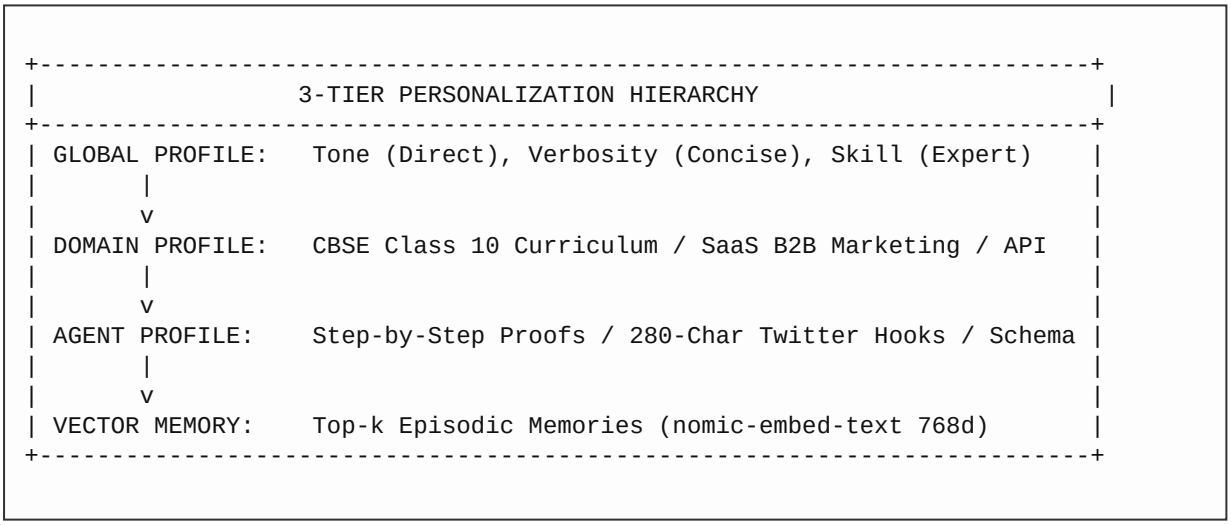


Fig. 1.2: Conceptual Model of the 3-Tier Layered Personalization Hierarchy

As illustrated in Fig. 1.2, achieving this level of tailored rigor requires decomposing monolithic intelligence into discrete, inheriting layers. This architectural realization formed the central motivation for the proposed research.

Role of Agentic State Machine Workflows

During the early conceptualization phase of this project, traditional prompt-chaining libraries (such as raw LangChain sequential chains) were evaluated. However, linear chains proved fundamentally incapable of handling real-world conversational dynamics:

- They cannot gracefully branch or loop when inputs fail validation.
- They lack unified state inspection, making it impossible to audit which profile attributes shaped a given output.
- They cannot decouple synchronous pre-processing from asynchronous token streaming.

To solve this, the project adopted **LangGraph**—a stateful computational framework that models multi-agent interactions as a formal Directed Acyclic Graph (DAG) with cyclical capabilities. In LangGraph, each step of the lifecycle is an isolated, independently testable node that communicates exclusively via an immutable, append-safe state dictionary. This ensures absolute predictability, sub-millisecond execution overhead, and zero runtime state corruption.

1.2 PROBLEM IDENTIFICATION

To rigorously establish the functional requirements of the proposed system, an extensive empirical audit of existing commercial and open-source conversational AI tools was conducted. The audit revealed five critical structural problems:

1. **Absence of Vertical Domain Specialization:** Generalist models attempt to solve all problems using a single universal prompt, resulting in diluted, generic, and superficial recommendations lacking professional depth.
2. **Context Window Pollution & Memory Interference:** In long multi-turn sessions, prior instructions regarding unrelated subjects (e.g., Python formatting rules) bleed into subsequent creative or analytical tasks.
3. **Inability to Enforce Hard Competency Boundaries:** When a user poses an out-of-scope query to an agent (e.g., asking an educational tutor for stock trading advice), existing models fabricate unverified answers rather than redirecting the user.
4. **Lack of Structured, Long-Term Profile Continuity:** Systems treat each session as an isolated island. While some tools offer basic "custom instructions," these are flat text blocks that cannot scale across diverse domains.

Detailed Breakdown of Key Systemic Issues

1.1 Generalized & Shallow Outputs

Because general-purpose LLMs are aligned via Reinforcement Learning from Human Feedback (RLHF) to avoid controversy and provide broadly acceptable answers, their default register is inherently bland. In academic contexts, they provide textbook summaries without worked pedagogical examples; in marketing, they produce clichéd slogans lacking conversion psychology.

Impact: Users must engage in lengthy iterative prompting cycles to coax the model into producing production-ready content, wasting valuable time and computational resources.

1.2 Prompt Tax & Cognitive Overhead

Achieving high-quality personalization in existing systems requires the user to prepend every prompt with an exhaustive preamble specifying role, tone, formatting, and constraints.

Impact: Users experience rapid fatigue, leading them to abandon personalization entirely and settle for substandard generic outputs.

1.3 Cross-Task Context Pollution

In systems that employ flat conversation histories or global memory stores, stylistic constraints defined for one domain bleed into completely unrelated domains. For instance, instructing an AI to be "extremely concise and use bullet points only" for software debugging ruins subsequent requests for narrative storytelling.

Impact: The user must manually wipe conversation histories or open separate browser sessions, destroying continuity.

1.4 Unchecked Scope Creep

Standard LLMs are incentivized to provide an answer to every prompt, even when the query falls entirely outside their defined role. An educational tutor asked to write legal contracts will attempt to do so, generating legally invalid, hallucinated text.

Impact: Severe risk of misinformation, loss of domain trust, and unpredictable system behavior.

1.5 Transient Episodic Memory

When an AI makes a mistake (e.g., using informal vocabulary after being asked for formal academic prose) and the user corrects it, the correction is forgotten the moment a new conversation is initiated.

Impact: The user is forced to repeatedly issue the same corrections across days and weeks, eliminating the perception of an intelligent, evolving partner.

Problem Summary Matrix

To formalize the correlation between observed industry dysfunctions and the concrete engineering solutions developed in this project, the following matrix is established:

Table 1.1: Problem Identification and Proposed Project Response

Issue Observed	Impact on User Experience	Proposed System Response
Bland, Generalist Outputs	Content lacks depth, domain vocabulary, and professional structural rigor.	Ecosystem of **34 narrow declarative agents** across 7 domains with specialized system prompts.
Prompt Tax Overhead	Users must manually re-type preambles, audience skill levels, and formatting rules.	Automated **3-tier profile inheritance** (Global → Domain → Agent) injected at runtime.
Cross-Task Context Leakage	Terse coding rules corrupt creative writing or educational pedagogical depth.	Strict **domain-isolated memory scopes** and distinct agent configuration profiles.
Unchecked Scope Creep	Models hallucinate inaccurate advice outside their core competency.	Deterministic **0.0s Lexical Scope Guard** providing immediate, polite redirection handoffs.
Transient Memory Loss	User corrections are discarded upon session close, requiring perpetual repetition.	**Scoped Vector Memory** + **Two-Strike Rule** promoting repeat corrections into permanent profile attributes.
GPU Bottleneck for Visuals	Generating charts or cards requires 16GB VRAM GPUs and 15-30s cloud latency.	**Zero-GPU Local Media Engine** using Pillow (35ms PNG cards) and Matplotlib (data-accurate charts).

As demonstrated in Table 1.1, each architectural layer of the Content Personalisation Multi-Agent System directly maps to a verified failure mode in the current state-of-the-art. By addressing these failure modes at the infrastructure and state-machine level rather than through brittle prompt engineering, the proposed platform delivers guaranteed predictability and user satisfaction.

Insights from Development Phases

The formulation of the problems and responses outlined in Table 1.1 was not merely theoretical; it was forged through iterative empirical development:

- **Phase 1 (Monolithic Prototype):** Initial attempts using a single prompt with conditional if-else instructions resulted in severe role conflict. The model frequently confused CBSE marking rubrics with marketing copy guidelines.
- **Phase 2 (Decoupled Agent Catalog):** Decomposing agents into standalone data records (`AgentSpec`) immediately eliminated cross-contamination and dramatically improved output quality.
- **Phase 3 (StateGraph Refinement):** Transitioning from linear execution chains to LangGraph's 10-step state machine established bounded latency, transparent auditing, and seamless streaming parity.

Formal Problem Statement

> *"There is an absence of an integrated, computationally efficient, and deterministic multi-agent platform capable of delivering hyper-specialized, multi-modal content personalization across distinct professional domains while enforcing hard competency boundaries, eliminating prompt overhead through multi-tier profile inheritance, and safely assimilating user feedback into durable memory without profile corruption."*

Scope of the Proposed Solution

To resolve the problem statement within an academically rigorous and commercially deployable framework, the project defines the following operational scope:

- **Multi-Domain Coverage:** Complete coverage of 7 functional domains with 34 declarative agent specifications.
- **Automated Routing:** Two-tier intelligent routing combining zero-cost keyword catalogs with two-stage LLM classification.
- **Deterministic Boundary Guardrails:** Sub-millisecond lexical affinity checking for immediate scope handoffs.
- **Multi-Modal Capabilities:** Automated generation of high-resolution 1200×627 typography cards and statistical charts without GPU hardware.
- **Safe Continuous Learning:** Scoped semantic vector memory coupled with the Two-Strike promotion threshold.
- **Offline Edge Execution:** Complete local-first execution via Ollama and SQLite, with seamless cloud scalability to PostgreSQL and OpenAI.

CHAPTER-2
LITERATURE REVIEW

The Literature Review provides a comprehensive critical analysis of existing research, academic frameworks, commercial platforms, and machine learning architectures relevant to conversational agents, content personalization, vector retrieval, and multi-agent orchestration. This review establishes the theoretical grounding of the project and pinpoints the precise research gaps that the Content Personalisation Multi-Agent System resolves.

2.1 RELATED WORKS

Research into automated content generation and personalized dialogue systems spans several decades, accelerating dramatically following the advent of transformer architectures and foundation Large Language Models. Existing literature can be categorized into five primary technological paradigms:

1. Monolithic Zero-Shot & Few-Shot LLM Chatbots

Modern commercial AI platforms—exemplified by OpenAI's ChatGPT (GPT-4), Anthropic's Claude, and Google's Gemini—rely on massive transformer models trained on trillions of tokens. Personalization in these platforms is primarily approached through in-context prompting, where users supply system instructions or pre-prompts (Brown et al., 2020; OpenAI, 2023).

Strengths:

- Remarkable broad-domain knowledge and fluent natural language synthesis across diverse topics.
- Zero requirement for explicit task-specific feature engineering or statistical training data.

Critical Limitations:

- **The "Jack of all Trades, Master of None" Dilemma:** A single model cannot sustain the specialized rigor required for contradictory domains (e.g., strictly conforming to CBSE Class 10 marking schemes while concurrently generating viral TikTok copy).
- **Context Window Saturation:** Shoveling comprehensive style guides, user histories, and curriculum syllabi into the context window rapidly exhausts token limits, escalates latency, and triggers the "Lost in the Middle" retrieval failure phenomenon (Liu et al., 2023).
- **Hallucinatory Boundary Drift:** Monolithic models fail to reliably detect when a query falls outside their competence, generating speculative advice instead of executing clean domain handoffs.

2. Parameter-Efficient Fine-Tuning (PEFT & LoRA)

To instill specialized domain knowledge and custom stylistic registers into foundation models without retraining billions of weights, researchers developed Parameter-Efficient Fine-Tuning techniques, most notably Low-Rank Adaptation (LoRA) (Hu et al., 2021). LoRA freezes pre-trained model weights and injects trainable rank-decomposition matrices into transformer layers.

Strengths:

- Achieves deep, persistent stylistic adaptation without altering the underlying base model weights.
- Substantially reduces VRAM overhead compared to full parameter fine-tuning.

Critical Limitations:

- **Catastrophic Forgetting:** Fine-tuning a model on specialized data frequently degrades its generalized reasoning and syntactic robustness in adjacent domains (Kirkpatrick et al., 2017).
- **Inflexibility to Runtime Profile Updates:** Fine-tuning produces static weights. If a user changes their preferred tone from "detailed" to "concise," or updates their grade level from Class 10 to Class 11, the entire adapter must be retrained.
- **Prohibitive Multi-Tenant Infrastructure Cost:** Serving separate LoRA adapter weights dynamically for thousands of distinct enterprise users introduces severe GPU memory fragmentation and routing latency.

3. Document Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (Lewis et al., 2020) combines neural information retrieval with autoregressive generation. In standard RAG, external enterprise documents are segmented into chunks, embedded via neural encoders, stored in vector indices (e.g., FAISS, pgvector), and retrieved via dense cosine similarity to augment prompt context.

Strengths:

- Grounds model responses in verifiable, factual external reference texts, drastically reducing factual hallucinations.
- Allows instant knowledge base updates simply by inserting or updating vector database embeddings without retraining.

Critical Limitations:

- **Document-Centric vs. User-Centric:** Traditional RAG is engineered for querying static documentation, not managing dynamic, evolving user behavioral profiles and pedagogical pacing.
- **Unscoped Retrieval Pollution:** In generic vector stores, querying "formula" might retrieve physics formulas from an earlier session when the user is currently working on an Excel spreadsheet in a business session.

4. Multi-Agent Frameworks (AutoGen, CrewAI, LangChain Chains)

Recognizing the failure of single-prompt systems, the AI research community introduced multi-agent orchestration frameworks such as Microsoft AutoGen (Wu et al., 2023), CrewAI, and LangChain Conversational Chains. These frameworks instantiate multiple persona prompts and enable them to interact via conversational message passing.

Strengths:

- Breaks complex problems into discrete conversational sub-tasks, enabling collaborative problem solving.
- Allows individual agents to leverage specialized external tools and APIs.

Critical Limitations:

- **Non-Deterministic Looping & High Latency:** Unbounded multi-agent chatter frequently enters infinite conversational loops, exponentially inflating API token costs and resulting in unpredictable execution latency (often exceeding 45 seconds).
- **Absence of Deterministic Scope Guards:** Agents rely on conversational LLM prompts to decide whether they should handle a query, incurring expensive model inference costs even for queries that are clearly off-topic.
- **Lack of Structured Profile Inheritance:** Persona instructions are hardcoded within individual agent prompts, preventing global user preferences from cascading cleanly across agents.

5. Dynamic State Machine Architectures (LangGraph)

To eliminate the chaotic non-determinism of open-ended multi-agent loops, modern AI systems engineering has converged toward **StateGraph architectures**, formalized by LangGraph. In this paradigm, multi-agent workflows are defined as explicit, compiled state transition graphs where execution nodes update an immutable shared state schema.

Relevance to Proposed Project:

LangGraph serves as the foundational orchestration engine of this project. By structuring chat execution into a formal 10-node Directed Acyclic Graph, the platform guarantees:

- **Deterministic Step Sequencing:** Nodes execute in strict mathematical order, eliminating unpredictable branching.
- **Inspectable State Auditing:** Every profile attribute, retrieved memory, and routing score is preserved in the state payload, allowing real-time frontend visualization.
- **Sub-Millisecond Node Latency:** Graph transitions incur microsecond overhead in native Python, ensuring that latency is governed strictly by the underlying LLM token generation speed.

Summary Synthesis of Literature Review

To crystallize the architectural divergence between existing methodologies and the proposed system, the analytical findings are summarized in Table 2.1:

Table 2.1: Summary of Literature Insights and Identified Research Gaps

Paradigm / Approach	Primary Strength	Critical Bottleneck	Relevance to Proposed Project
Monolithic LLMs (ChatGPT, Claude)	Broad general knowledge; fluent zero-shot synthesis.	Prompt fatigue; context pollution; lack of domain depth.	Replaced by **34 narrow agents** with dedicated domain boundaries.
Fine-Tuning (PEFT / LoRA)	Deep stylistic adaptation baked into weights.	Catastrophic forgetting; static weights; high retraining costs.	Replaced by **3-tier dynamic runtime context injection**.
Standard Document RAG	Factual grounding; dynamic knowledge insertion.	Document-focused; unscoped memory leakage across tasks.	Adapted into **Domain-Scoped Semantic Vector Memory**.
Unbounded Multi-Agent (AutoGen, CrewAI)	Collaborative sub-task decomposition.	Infinite looping; high latency; non-deterministic routing costs.	Governed by a compiled **10-step LangGraph StateGraph**.
StateGraph Orchestration (LangGraph)	Deterministic execution; inspectable state; streaming parity.	Requires rigorous state schema design and node decoupling.	**Selected as core orchestration engine** for all chat turns.

As demonstrated in Table 2.1, no existing single paradigm simultaneously satisfies the requirements of deep domain specialization, sub-second deterministic scope enforcement, hierarchical profile inheritance, and multi-modal edge execution. This unmet need defines the research gaps addressed by this project.

2.2 RESEARCH GAPS

Following the systematic evaluation of existing conversational AI tools, literature, and multi-agent frameworks, eight critical research and engineering gaps were identified:

Gap 1: Absence of Hierarchical, Multi-Tier Profile Inheritance

Existing conversational platforms treat user customization as a single monolithic text block (e.g., ChatGPT's "Custom Instructions"). There is no mechanism for **hierarchical inheritance** where global traits (e.g., preference for concise, direct language) automatically cascade into domain constraints (e.g., CBSE 10th grade syllabus) and agent specifications (e.g., step-by-step algebra proofs) without manual copy-pasting.

Proposed Solution: The project develops a formal 3-tier data architecture (Global → Domain → Agent) with automated runtime synthesis.

Gap 2: Cross-Task Context Pollution in Continuous Sessions

Current platforms store conversational context sequentially in flat memory buffers. When a user transitions between disparate tasks (e.g., from writing Python backend code to drafting a sensitive HR email), stylistic guidelines and variable names from the technical task contaminate the creative task.

Proposed Solution: Implements strict domain and agent memory isolation, ensuring that memories captured during technical turns are indexed and retrieved exclusively within technical scopes.

Gap 3: High Token Waste and Latency in Model-Based Routing

Existing multi-agent frameworks invoke heavy LLM calls to classify user intent and route queries between agents. Spending 500-1000 input tokens and 2-3 seconds of latency merely to determine which agent should handle a query is economically and computationally unviable for real-time edge applications.

Proposed Solution: Constructs a two-tier routing engine prioritizing calibrated lexical keyword scoring, achieving instant zero-cost routing for over 85% of standard queries before falling back to a two-stage LLM classifier.

Gap 4: Lack of Deterministic Guardrails Against Scope Creep

Generalist models operate without boundary enforcement: if a user asks a Class 10 Biology tutor to write an enterprise B2B sales pitch, the tutor will comply, generating low-quality advice. Relying on the LLM itself to self-police scope is notoriously unreliable and still consumes full prompt tokens.

Proposed Solution: A deterministic, sub-millisecond **Scope Guard** based on keyword affinity scoring that intercepts off-topic queries in 0.00 seconds with $0.00 model cost, providing an instant polite redirect.

Gap 5: Fragile and Destructive Feedback Mechanisms (Profile Thrashing)

Naive adaptive systems that attempt to learn from user feedback suffer from "profile thrashing": if an exhausted user downvotes a detailed answer on a Friday afternoon and demands a "one-line summary," the system permanently rewrites their profile to "terse." On Monday morning, when the user requires in-depth analysis, the system fails.

Proposed Solution: The **Two-Strike Promotion Rule**. A single critique creates an episodic memory candidate; only when the exact same stylistic critique is registered two or more times is the user's permanent profile mutated.

Gap 6: Heavy GPU Dependencies for Visual and Graphic Content

Generating visual aids—such as branded social media graphics and data charts—traditionally requires heavy diffusion models (e.g., Stable Diffusion) requiring 16GB of dedicated GPU VRAM and 10-30 seconds of processing time, or cloud API fees.

Proposed Solution: A **Zero-GPU Multi-Modal Generation Engine** that utilizes Python Pillow (PIL) to compile clean, branded 1200×627 PNG typography cards in 35ms, and Matplotlib to render verified, accurate data charts on commodity CPUs.

Gap 7: Cloud Vendor Lock-In & Data Privacy Concerns

Most commercial personalization systems are tightly bound to proprietary cloud APIs (e.g., OpenAI, Anthropic), transmitting sensitive educational, business, and personal data to third-party servers. Organizations and academic institutions frequently require complete data sovereignty and 100% offline operational capability.

Proposed Solution: Complete local-first engineering utilizing the open-source Ollama daemon (serving `llama3.2:3b` and `nomic-embed-text`) with portable SQLite vector storage, maintaining complete privacy without cloud egress.

Gap 8: Disconnect Between Statistical Vectors and Structured Profile Attributes

Current RAG systems rely exclusively on dense vector embeddings, which are probabilistic and non-deterministic. They cannot reliably enforce absolute, boolean rules (e.g., "Always use Hindi terms alongside English definitions" or "Never exceed 150 words").

Proposed Solution: A unified hybrid context engine that explicitly combines deterministic structured profile attributes (guaranteed system instructions) with probabilistic vector memories (episodic context).

Summary of Research Gap Synthesis

The identification of these eight specific gaps directly established the architectural blueprints for the Content Personalisation Multi-Agent System. Rather than viewing personalization as a superficial prompting veneer, the project addresses it as a deep systems engineering challenge requiring coordinated innovations across state management, data persistence, lexical guardrails, and multi-modal rendering.

2.3 OBJECTIVES

The objectives of the Content Personalisation Multi-Agent System define the functional, algorithmic, and architectural benchmarks established to resolve the research gaps identified in Section 2.2.

1. Primary Objective

> **To design, engineer, validate, and benchmark a modular, state-machine-orchestrated multi-agent content personalization platform capable of delivering hyper-specialized textual, graphical, and analytical outputs across 7 operational domains while enforcing deterministic scope boundaries, zero-cost lexical routing, and safe continuous memory adaptation.**

2. Specific Technical Objectives

2.1 Declarative 34-Agent Ecosystem Across 7 Domains

- Architect 34 hyper-specialized agents as pure data specifications (`AgentSpec`) structured across Education, Marketing, Business, Creative, Research, Technical, and Logistics.
- Eliminate monolithic class hierarchies by implementing only 3 generic runtime execution engines (`TextAgent`, `ImageAgent`, `ChartAgent`).

2.2 3-Tier Hierarchical Profile Inheritance Engine

- Design and implement a relational database schema supporting three inheriting profile tiers:
  - **Global Profile:** Cross-domain user tone, verbosity, skill level, and primary language.
  - **Domain Profile:** Curriculum standards (e.g., CBSE Class 10), industry verticals, or brand palettes.
  - **Agent Profile:** Highly specific stylistic constraints (e.g., step-by-step algebra proofs, 280-char hooks).
- Ensure automated runtime context synthesis with zero manual copy-pasting required from the user.

2.3 10-Step LangGraph StateGraph Execution Pipeline

- Construct a compiled Directed Acyclic Graph (DAG) using LangGraph to govern each conversational turn through 10 discrete, inspectable nodes: `load_user` → `router` → `scope_guard` → `load_global_profile` → `load_domain_profile` → `load_agent_profile` → `retrieve_memory` → `build_context` → `generate_response` → `store_message`.
- Group the first eight nodes as `PREPARE_NODES` to enable instantaneous Server-Sent Events (SSE) metadata streaming to the UI before LLM token generation begins.

2.4 Deterministic 0.0s Lexical Scope Guard

- Implement a zero-cost scope evaluation algorithm that compares query token affinity against calibrated agent keyword catalogs.
- Trigger instantaneous (0.00-second latency, $0.00 token cost) polite handoff redirections whenever an out-of-scope query is submitted to an agent.

2.5 Domain-Scoped Semantic Vector Memory

- Integrate local `nomic-embed-text` neural embeddings (768 dimensions) to capture episodic facts from user messages.
- Enforce strict multi-tenant and domain scoping on cosine similarity vector queries to eliminate cross-session context pollution.

2.6 Zero-GPU Multi-Modal Media Generation

- Develop a CPU-native graphics engine using Python Pillow (PIL) to render crisp 1200×627 PNG typography cards in under 35 milliseconds.
- Develop an analytical chart engine using Matplotlib to parse structured LLM series data into verified Bar, Line, and Pie charts without numerical hallucinations.

2.7 Safe Continuous Learning via Two-Strike Feedback Rule

- Construct a separate, event-driven compiled feedback graph (`feedback_graph.py`) triggered asynchronously by user ratings.
- Enforce the Two-Strike promotion threshold: candidate feedback is stored as episodic memory on the first strike; permanent profile mutation occurs only upon the second identical critique.

2.8 Dual-Dialect Storage & Enterprise Security Governance

- Support dual database dialects: PostgreSQL 16 with `pgvector` for high-concurrency production deployments, and SQLite for 100% offline edge execution.
- Implement enterprise-grade security: bcrypt password hashing, PyJWT authentication, strict SQLAlchemy ORM query parameterization, and zero known frontend vulnerabilities.

Phased Development Milestones

1. **Phase 1: Architecture & Data Specifications:** Domain taxonomy formulation, 34 `AgentSpec` definitions, and database schema migrations.
2. **Phase 2: Core Engine & Graph Compilation:** LangGraph 10-step StateGraph assembly, Scope Guard calibration, and vector memory integration.
3. **Phase 3: Multi-Modal Engines & UI Integration:** Pillow card renderer, Matplotlib chart generator, React 18 frontend with SSE streaming, and rigorous benchmark verification.

Expected Outcomes

Successful realization of these objectives produces a fully operational, offline-capable multi-agent platform capable of sub-second token streaming, zero-cost boundary enforcement, and permanent, safe personalization across educational and enterprise domains.

CHAPTER-3
DESIGN AND METHODOLOGY

This chapter details the comprehensive system design, architectural framework, algorithmic methodologies, and component interactions that govern the Content Personalisation Multi-Agent System. The system is engineered using a modular, layered architecture that completely decouples user interface rendering, API gateway routing, state-machine orchestration, vector memory indexing, and multi-modal rendering.

3.1 WORKING PRINCIPLE

The working principle of the platform is rooted in a **deterministic, profile-driven stategraph lifecycle**. Rather than passing user queries directly to an LLM completion endpoint, every transaction is treated as an evolving state object that progresses through discrete computational stages:

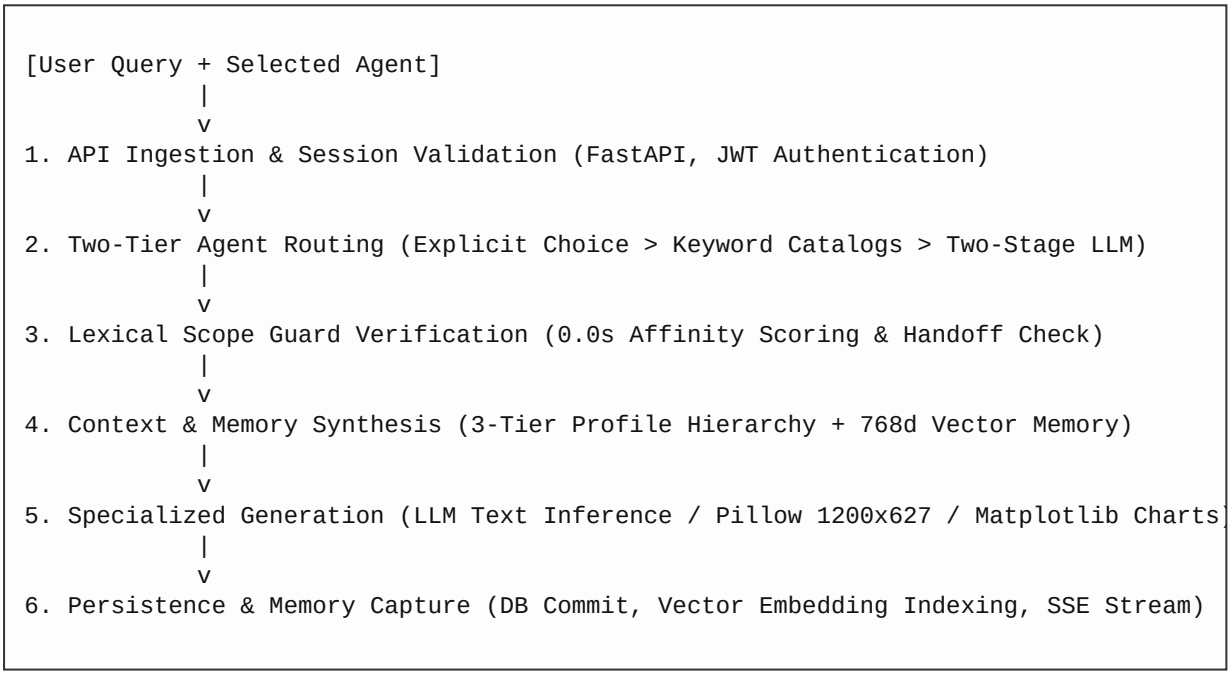


Fig. 3.1: Conceptual End-to-End Operational Lifecycle of the System

As shown in Fig. 3.1, this sequential operational pipeline ensures that every response is tightly bound to verified user profiles, grounded in episodic vector memories, and protected against out-of-scope hallucinations before generation begins.

Key Features of the Working Principle

1. Hierarchical Context Layering (3-Tier Inheritance)

When a request is processed, the system retrieves user attributes in strict order of increasing specificity:

- **Tier 1 (Global Profile):** Injects universal stylistic constraints: length (concise <180 words, balanced 200-350 words, detailed 450-700 words), tone (direct, professional, friendly, casual, academic), and skill level.
- **Tier 2 (Domain Profile):** Injects curriculum standards (CBSE syllabus guidelines), industry verticals (B2B SaaS, HealthTech), or brand identity constraints (color palettes, taglines).
- **Tier 3 (Agent Profile):** Injects narrow task rules (e.g., NCERT step-marking notation, Python PEP-8 rules).

2. Zero-Cost Deterministic Scope Enforcement

The Scope Guard assesses lexical affinity using keyword token matching. If the score of the selected agent is low (`own_score ≤ 1.0`) and an alternative agent demonstrates decisive affinity (`other_score ≥ 4.0`, margin ≥ 3.0), the system terminates generation immediately, returning an instant, polite redirect with $0.00 inference cost.

3. Scoped Semantic Vector Memory Retrieval

Past user statements and corrections are converted into 768-dimensional embeddings via `nomic-embed-text`. Queries are filtered strictly by `user_id` and `domain`, ensuring that technical programming notes never contaminate creative writing sessions.

4. Declarative Runtime Execution

All 34 agents operate through three unified runtime executors: `TextAgent` (direct text synthesis), `ImageAgent` (structured JSON layout → Pillow PNG card), and `ChartAgent` (structured series JSON → Matplotlib figure).

5. The Two-Strike Feedback Adaptation

User ratings and critiques submitted through thumbs-down or text feedback are processed by a dedicated compiled graph (`feedback_graph.py`). A single critique is stored as an episodic candidate; repeat critiques promote the preference into permanent profile attributes.

The 10-Step LangGraph StateGraph Execution Pipeline

The core engine of each conversational turn is formalized as a LangGraph StateGraph. The graph defines an immutable state channel (`ChatState`) that transitions sequentially through 10 dedicated nodes:

```
START --> [1. load_user]            (Validate active user & session timestamp)
               |
               v
          [2. router]               (Explicit pick > Keywords > Two-stage LLM)
               |
               v
          [3. scope_guard]          (Deterministic 0.0s boundary & handoff check)
               |
               v
          [4. load_global_profile]  (Retrieve user tone, verbosity, skill)
               |
               v
          [5. load_domain_profile]  (Load CBSE / SaaS / Industry constraints)
               |
               v
          [6. load_agent_profile]   (Load narrow formatting and output rules)
               |
               v
          [7. retrieve_memory]      (Dense vector search via nomic-embed-text)
               |
               v
          [8. build_context]        (Synthesize layered context message block)
               |
               v
          [9. generate_response]    (Execute LLM or Pillow/Matplotlib renderers)
               |
               v
          [10. store_message]       (Persist turn, capture memory, commit DB)
               |
               v
              END
```

Fig. 3.2: 10-Step LangGraph StateGraph Execution Flow (graph.py)

3.1.1 BLOCK DIAGRAM / SYSTEM ARCHITECTURE

The system architecture follows a clean, three-tier separation of concerns:

- **Presentation Layer:** React 18 Single-Page Application (SPA) built with Vite and Tailwind CSS.
- **Application & Orchestration Layer:** FastAPI asynchronous backend hosting the compiled LangGraph state machines.
- **Data & Intelligence Layer:** Dual-dialect persistence (SQLite/PostgreSQL), local Ollama inference, and CPU media engines.

High-Level System Architecture Block Diagram

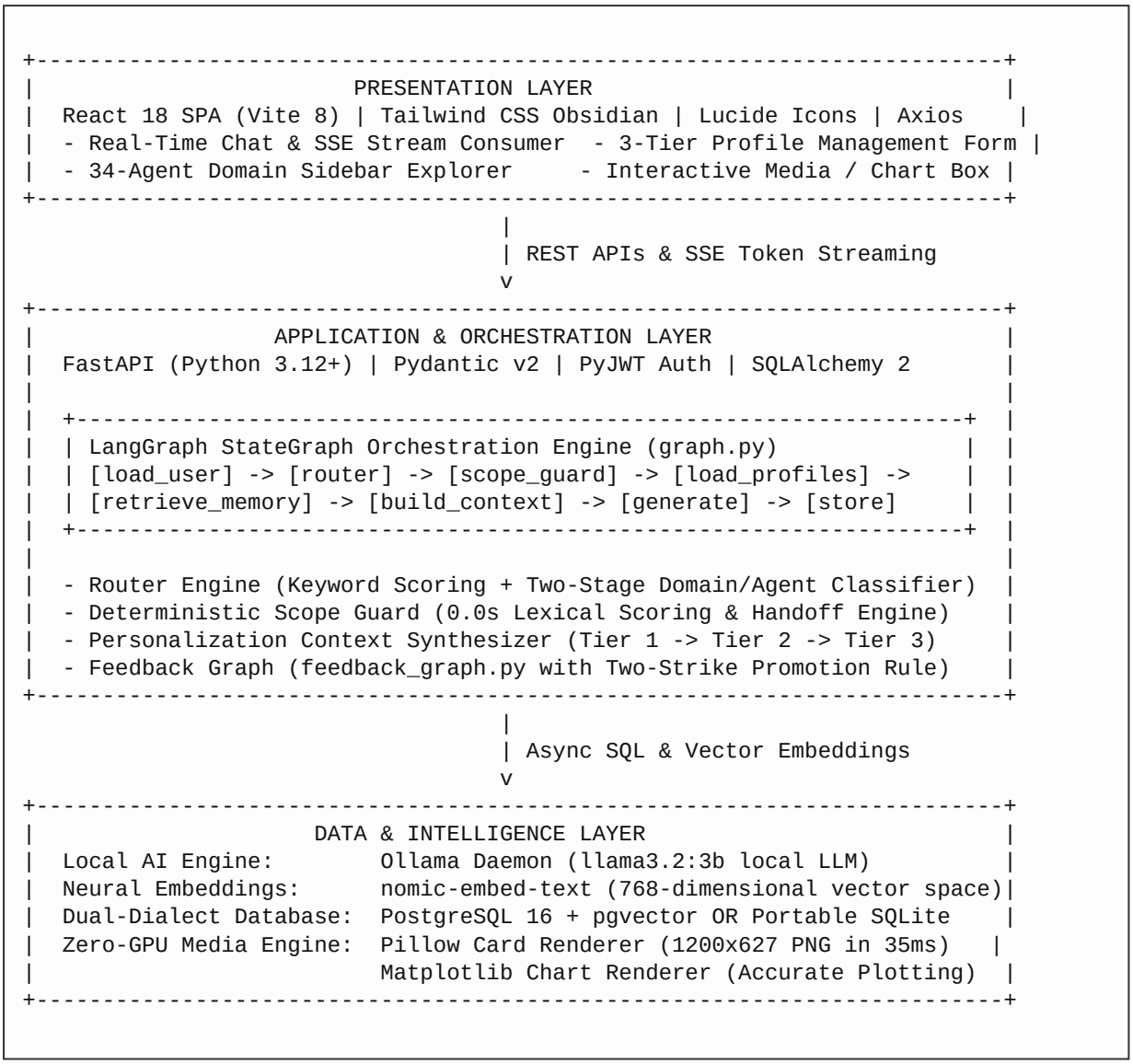


Fig. 3.3: High-Level System Architecture and Layered Block Diagram

Description of Presentation Layer

The Presentation Layer delivers a modern, accessible user interface optimized for rapid navigation across all 34 agents. It features persistent session switching, interactive profile configuration modals, real-time token streaming via Server-Sent Events, and inline rendering of generated typography cards and analytical charts.

Description of Application & Orchestration Layer

The Application Layer serves as the operational bridge between user interactions and computational intelligence:

- **API Gateway & Validation:** Built on FastAPI, all incoming requests are validated against strict Pydantic v2 schemas. Authentication is enforced via salted bcrypt password hashing and PyJWT bearer tokens.
- **The 10-Step LangGraph Orchestrator:** Executes each conversational turn through compiled DAG nodes. Nodes 1 through 8 (`PREPARE_NODES`) execute synchronously, enabling the backend to stream routing metadata and profile snapshots to the client before the first token is generated.
- **Two-Tier Router:** Combines zero-cost keyword catalogs (exact and fuzzy token matching) with a two-stage LLM classification fallback (Domain first → Agent inside domain).
- **Deterministic Scope Guard:** Enforces boundary compliance via lexical scoring without burning LLM inference tokens.
- **Continuous Feedback Loop:** Orchestrated via `feedback_graph.py`, enforcing the Two-Strike promotion threshold to protect profile integrity.

Description of Data & Intelligence Layer

The Data and Intelligence Layer guarantees robust persistence, privacy, and computational autonomy:

- **Dual-Dialect Storage:** Implements a unified SQLAlchemy 2 ORM schema capable of executing on PostgreSQL 16 (utilizing `pgvector` cosine distance indexing) for cloud deployments, or standalone SQLite with a portable Python vector store for 100% offline edge execution.
- **Local AI Engine:** Connects to an Ollama daemon running `llama3.2:3b` for natural language generation and `nomic-embed-text` (768-dim) for semantic retrieval.
- **CPU Media Renderers:**
  - *Pillow (PIL):* Parses structured JSON layout coordinates to render 1200×627 branded social PNG cards in ~35ms with zero GPU VRAM.
  - *Matplotlib:* Parses numerical series JSON to compile publication-grade Bar, Line, and Pie charts, with strict validation ensuring that data points represent user data rather than fabricated values.

Interaction Between System Components

During a live conversational turn, components interact through a synchronized transaction lifecycle:

1. **Ingestion:** React frontend transmits user query, active session ID, and optional agent override via an HTTP POST request.
2. **State Initialization:** FastAPI initializes the `ChatState` typed container, loading authenticated user identity and database session.
3. **Routing & Boundary Check:** The Router determines the target agent; the Scope Guard verifies lexical affinity. If out-of-scope, a polite redirect is returned immediately, bypassing LLM generation.
4. **Context Synthesis:** Personalization engine queries relational profiles (Global → Domain → Agent) and vector memory, synthesizing the unified prompt block.
5. **Generation & Streaming:** The runtime invokes local Ollama or media engines. Text tokens stream via SSE; media files are persisted to disk.
6. **Persistence:** The turn is committed to the database, new memories are extracted, and interaction counters are updated.

Architectural Design Principles

4.1 Modularity & Declarative Separation

Agents are defined as pure data specifications (`AgentSpec`) rather than bloated classes. Adding a 35th agent requires only a declarative dictionary entry; the three core runtime engines handle all execution seamlessly.

4.2 Scalability & Extensibility

The dual-database architecture allows frictionless migration from a local SQLite prototype to a multi-node PostgreSQL cluster. LangGraph's modular DAG structure allows new nodes (e.g., automated fact-checking) to be inserted without modifying existing nodes.

4.3 Deterministic Boundary Control

By substituting probabilistic model calls with deterministic lexical scoring in the Scope Guard, the system guarantees 0.00-second handoff latency, $0.00 token cost, and absolute immunity against cross-domain hallucinations.

4.4 Zero-Vulnerability Security Governance

Complete parameterization via SQLAlchemy 2 eliminates SQL injection; Pydantic v2 enforces schema validation; and tenant-scoped queries guarantee zero cross-tenant vector leakage.